

Beyond the Chatbox

Egor Kotov

Contents

	Introduction	5
	Overview	5
	Workshop Details	5
	Before the Workshop: Accounts & Setup	6
	1. Recommended Accounts to Register	6
	2. Coding Agents We Will Use (and Why)	6
1	Introduction to LLM Agents	9
1.1	What is an LLM Agent?	9
1.1.1	Key Characteristics	9
1.2	Why Use Agents for Research?	9
1.3	Models	9
1.4	Next Steps	9
2	Environment Setup	11
2.1	 Launching the Environment	11
2.1.1	 Auto-Pause & Resume	12
2.2	 Managing Your Session & Quotas	12
2.2.1	Monthly Free Quotas (2026)	12
2.2.2	Checking Remaining Quota	12
2.2.3	Stopping Your Session Manually	12
2.3	Next Steps	13
I	Basics	
3	Agent Interaction	17
3.1	The Interaction Loop	17
3.2	Control vs. Autonomy	17
4	Exercise 1: Hello, Agent	19
4.0.1	The Goal	19
4.0.2	Steps	19
II	Advanced	
5	Security & Secrets	23
5.1	Common Risks	23

5.1.1	Unsafe Code Execution	23
5.1.2	Data Leakage	23
5.2	Mitigating Data Leakage	23
6	Exercise 2: The Vault	25
6.0.1	The Goal	25
6.0.2	Steps	25
7	The Workshop Workspace	27
7.1	Workspace Structure	27
7.2	The Sandbox Boundary	27
7.3	Next Steps	27
8	Environments & Skills	29
8.1	Managing Complex Dependencies	29
9	Exercise 3: The Pro Resident	31
9.0.1	The Goal	31
9.0.2	Steps	31
	References	33
	Production tools	33
	Website libraries	33
	Local project code	34
	Content license	34

Introduction

 Warning

THIS IS STILL WORK IN PROGRESS - EXPECT BROKEN LINKS AND MISSING CONTENT



Figure 0.1: © Image generated by Google Gemini | Nanobanana

Overview

This interactive tutorial and workshop provides a practical introduction to using large language models (LLMs) in coding agents, featuring applied examples in population research. It is organized by [Egor Kotov](#) with support from the [Max Planck Institute for Demographic Research](#).

Workshop Details

- **Date:** June 03, 2026
- **Time:** 13:30-16:30
- **Location:** Complesso Belmeloro, room “Lab O”, [Via Beniamino Andreatta, 8, 40126 Bologna BO, Italy](#)

Abstract

The software industry has been enjoying the benefits of coding agents for over a year now, with even top software engineers now having AI assistants generate a vast majority of their code. Now, the tools for agentic coding are mature enough for anyone to try.

Unlike many educational materials in this field, this workshop will also address the new vectors of attack that novice users might expose themselves to by using coding agents blindly. We will cover key security risks associated with agentic systems, including data leakage, unsafe code execution, and unintended modifications to analysis code and research materials. To mitigate these risks, we will introduce accessible strategies such as containerization, sandboxing, and workflow isolation.

Participants will also see how the agents themselves can assist in setting up and using these protective tools to improve transparency and reproducibility. The workshop will combine short demonstrations with applied examples relevant to population research. Participants are encouraged to bring their own projects or ideas, which we will use to explore how LLM agents can be integrated into real research workflows. For those without a project, guided exercises and example tasks will be provided.

Before the Workshop: Accounts & Setup

To make the most of our hands-on sessions, please complete the following steps before the workshop on **June 3, 2026**.

! Data Privacy & Security Warning

Free tiers for these LLM services typically **use your prompts and uploaded code for model training**. **Do not use these tools on private, sensitive, or proprietary research data!** Only use public or non-sensitive datasets during the workshop and for general testing.

1. Recommended Accounts to Register

Please sign up for the following services in advance to ensure you have enough free model usage quota during the exercises:

- **Google Account** (A personal/secondary account is perfect) — Used for **Gemini CLI** and **Antigravity CLI**. No credit card is required.
- **OpenCode Zen** (opencode.ai/zen) — Gives you free model access via the **OpenCode** agent. You can view the list of available models and details on [OpenCode Zen pricing/documentation](#).
- **OpenRouter** (openrouter.ai) — Allows accessing a wide variety of models. You'll need to generate a free **API Key** after registering, which can be used to access their catalog of **free models**.
- *Optional: NVIDIA NIM/Build* (build.nvidia.com) — Offers **free preview models** (note: account activation requires SMS verification via a mobile phone number).

i Note

If you have a subscription to another LLM service (such as OpenAI ChatGPT Plus, Claude Pro, Gemini Advanced, or Mistral) and prefer to use it during the workshop, please feel free to do so. However, our guided exercises will focus on the tools listed above to ensure everyone has access to the same models and features.

The skills you will learn here are highly transferable to other AI coding assistants, though setup steps and specific features may vary by provider. Please refer to the documentation of your chosen agent for setup instructions, and don't hesitate to ask the agent itself how to configure it for your desired models and features.

⚠ API & Account Safety Warning

Do **NOT** attempt to authenticate your Google or Anthropic (Claude) accounts directly within OpenCode. Their terms of service prohibit use of quota based subscriptions in third-party clients, and doing so can result in your accounts being permanently banned. OpenAI is currently a bit more flexible on this at the time of writing, but always verify provider policies.

2. Coding Agents We Will Use (and Why)

The coding agents are already pre-installed and configured inside the workshop's Docker container / Codespaces environment. We will focus on:

- **Google Antigravity CLI** (and its predecessor **Gemini CLI**)
 - *Why:* It provides a free daily quota to test cutting-edge models (like **Gemini 3.5 Flash**, **Gemini 3.1 Pro**, and **Claude 3.5 Sonnet**) directly from your Google account.

- **OpenCode CLI**

- *Why:* A flexible, open-source-friendly agent that can connect to multiple backends. We will use it with free models from **OpenCode Zen**, **OpenRouter**, and **NVIDIA NIM** to show how easy it is to switch between different model providers without vendor lock-in.

i Note

What is **CLI**? It stands for “Command Line Interface” and means that you will interact with the agent by typing commands in a terminal (similar to your R console), rather than through a graphical user interface (GUI). This allows for more flexibility and control over the agent’s behavior, as well as easier integration into coding workflows. This also gives you better insight into how the agent works under the hood, which is important for understanding its capabilities and limitations. You are of course free to use the same agents through their graphical interfaces too. We are also limited by the current tutorial environment and tools that are free to use. In our tutorial environment Gemini and Antigravity are only available as CLI, OpenCode is also CLI, but you will learn how to run it in graphical mode as well.

To get started with the workshop materials, please navigate through the sections in the navigation bar on the left or with pagination links in the bottom.

1. Introduction to LLM Agents

This section covers the basics of what LLM agents are and how they differ from standard chat interfaces.

1.1 What is an LLM Agent?

Unlike a standard LLM chat (like ChatGPT or Gemini web interface), an **agent** is a system that uses an LLM as its “reasoning engine” to perform tasks by interacting with the world.

1.1.1 Key Characteristics

- **Tool Use:** Agents can use tools such as search engines, calculators, or terminal commands.
- **Autonomy:** They can break down a complex goal into smaller steps and execute them sequentially.
- **Feedback Loop:** They can observe the output of a tool (e.g., a compiler error) and adjust their next action accordingly.

1.2 Why Use Agents for Research?

In academic research, agents can: - **Automate Data Cleaning:** Write and run scripts to fix inconsistencies in large datasets. - **Literature Review:** Search, summarize, and synthesize findings from hundreds of papers. - **Reproducible Analysis:** Generate documented code that follows best practices.

1.3 Models

See [models comparison](#)

1.4 Next Steps

Now that you know what agents are, let's look at [Security & Sandboxing](#).

2. Environment Setup

To ensure a consistent, secure, and hassle-free experience, we will use a pre-configured cloud-based development environment. We will use **GitHub Codespaces**.

💡 ⏳ Start the Setup Early and Focus on the Presentations!

First-time environment setup and container building can take **up to 10 minutes** to compile and prepare.

We highly recommend launching the codespace right at the beginning of the workshop. While the setup runs automatically in the background, you can continue to fully focus on the presentations, live demonstrations, and slides shown by the tutor.

2.1 🛠️ Launching the Environment

To launch the Codespaces environment manually from the GitHub interface, follow these steps:

1. Navigate to the [workshop repository](#).
2. Click the green **Code** button in the top right.
3. Select the **Codespaces** tab.
4. Click **Create codespace on main** (or click the **...** menu to configure machine size).

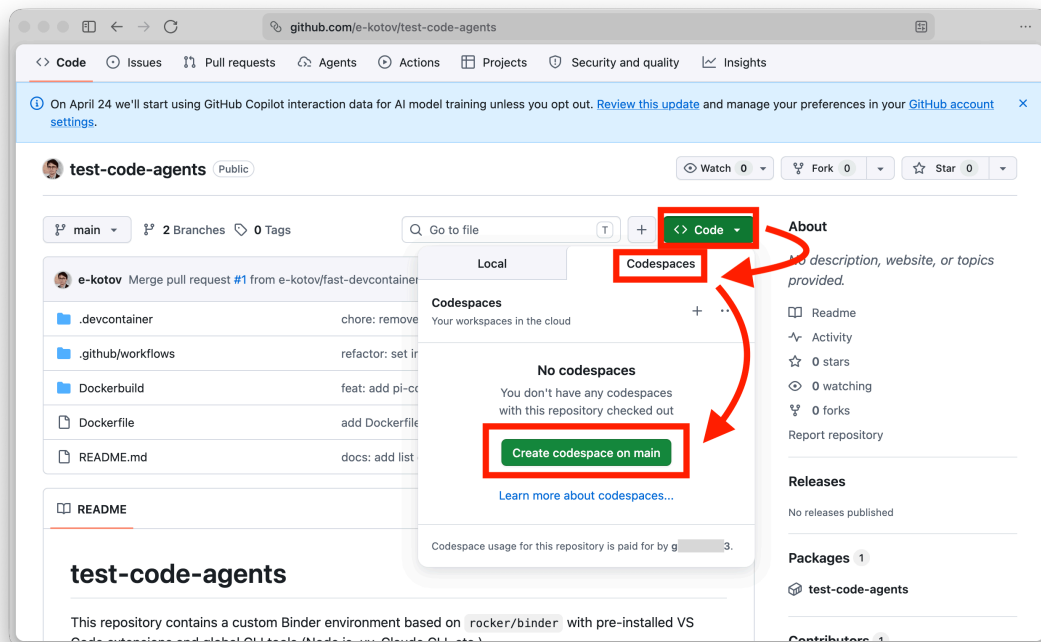


Figure 2.2: Launch in GitHub Codespaces

Free & Secure Environment

- **It is 100% Free:** GitHub provides a generous free tier of **60 hours** of Codespaces usage per month for every personal account. No credit card, billing info, or payment is required.
- **It is Completely Secure:** The environment runs inside a secure, isolated container (virtual machine) in the cloud. It does not touch your physical computer, install any local files, or run any software on your machine. It belongs to you and it is also mostly safe to enter your account data (e.g. Google account, especially if secondary or test account, API keys, other accounts for subscription services or services we will use during the workshop) without worrying about security or privacy issues.

2.1.1 Auto-Pause & Resume

To save your free hours, the environment automatically pauses after 30 minutes of inactivity. When you restart it, your files, open tabs, and terminal history are fully preserved—just like waking up a laptop from sleep (though you may need to restart any active terminal commands or running applications).

2.2 Managing Your Session & Quotas

GitHub Codespaces provides a generous free tier, but it is important to understand how usage is consumed to avoid running out of credits.

2.2.1 Monthly Free Quotas (2026)

The following quotas apply to personal accounts.¹

Account Type	Actual Usage Hours	Storage (per month)
GitHub Free	60 hours	15 GB
GitHub Pro	90 hours	20 GB

Storage Persistence Quota

Storage (15GB/20GB) is consumed even when the codespace is **stopped**. To save your storage quota, make sure to **delete** codespaces you no longer need at github.com/codespaces once the workshop is fully over.

2.2.2 Checking Remaining Quota

Tip

You can monitor your real-time usage and remaining balance at any time:

1. Go to your [GitHub Settings > Billing and plans](#).
2. Look for the **Codespaces** section under **Usage this month** or try this direct link: [GitHub Codespaces Usage](#).

2.2.3 Stopping Your Session Manually

When you are done with a session, you can stop the codespace manually to instantly stop consuming usage hours:

¹While the [official billing documentation](#) lists these as 120 and 180 **core hours**, the minimum instance size is 2 CPU cores. This means every 1 hour of real-time work consumes 2 core hours, resulting in the actual limits shown below.

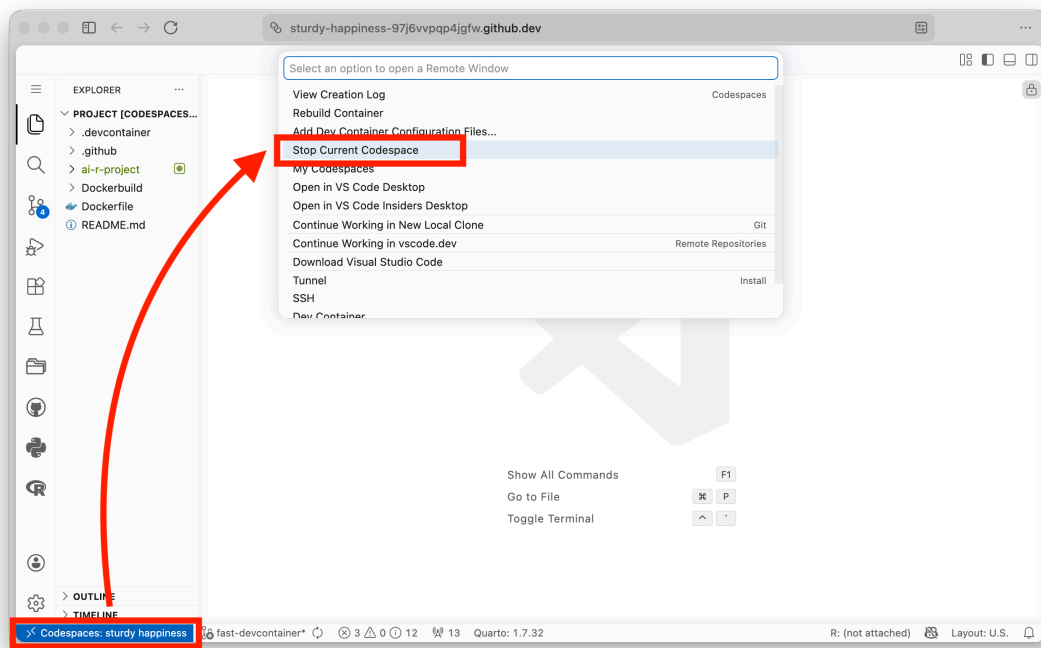


Figure 2.3: Stop GitHub Codespace

2.3 Next Steps

Now that your environment is ready, let's explore the [Workshop Workspace](#).



Basics

3	Agent Interaction	17
3.1	The Interaction Loop	17
3.2	Control vs. Autonomy	17
4	Exercise 1: Hello, Agent	19

3. Agent Interaction

3.1 The Interaction Loop

Working with coding agents involves a continuous loop: 1. **Prompt:** You provide a goal or question. 2. **Thought:** The agent formulates a plan. 3. **Action:** The agent uses tools (reading files, running bash commands, editing code). 4. **Response:** The agent presents the result or asks for clarification.

3.2 Control vs. Autonomy

You can choose how much autonomy to give your agent: - **Surgical Mode:** Ask for one specific change at a time. - **Goal Mode:** Provide a high-level goal and let the agent manage the steps.

Ready to try it out?

[Go to Exercise 1: Hello, Agent →](#)

4. Exercise 1: Hello, Agent

Let's get comfortable with the coding agent on a basic, familiar task.

Location: `userspace/projects/task-01-basic/`

4.0.1 The Goal

Use a built-in R dataset like `mtcars` or `iris` to perform some basic data manipulation and plotting.

4.0.2 Steps

1. Open your terminal and navigate to the `userspace/projects/task-01-basic/` folder.
2. Start your agent (Gemini CLI, Antigravity, or OpenCode).
3. Prompt the agent: *"Calculate the mean mpg for different cyl groups in the mtcars dataset and create a simple visualization using ggplot2."*
4. Watch the agent write the R script, execute it, and generate the plot.

Observe the Agent

Pay attention to how the agent decides to name files and whether it creates a separate script or runs code directly in the terminal.

II

Advanced

5	Security & Secrets	23
5.1	Common Risks	23
5.2	Mitigating Data Leakage	23
6	Exercise 2: The Vault	25
7	The Workshop Workspace	27
7.1	Workspace Structure	27
7.2	The Sandbox Boundary	27
7.3	Next Steps	27
8	Environments & Skills	29
8.1	Managing Complex Dependencies	29
9	Exercise 3: The Pro Resident	31
9.0.2	References	33
9.0.2	Production tools	33
9.0.2	Website libraries	33
9.0.2	Local project code	34
9.0.2	Content license	34

5. Security & Secrets

Using coding agents involves granting them the ability to execute code on your machine. This introduces several security and integrity risks that must be managed.

5.1 Common Risks

5.1.1 Unsafe Code Execution

An agent might generate code that deletes files or opens network vulnerabilities. Always use the **human-in-the-loop** pattern to review commands.

5.1.2 Data Leakage

Agents might send sensitive data (API keys, credentials) to the LLM provider. This is why we use safeguards like hooks and plugins.

5.2 Mitigating Data Leakage

To prevent the agent from reading sensitive files (like `.env` files), we use: - **OpenCode: Env Protection Plugins**. - **Gemini CLI: Hooks** to block access to specific files.

Let's put this into practice.

[Go to Exercise 2: The Vault](#) →

6. Exercise 2: The Vault

Let's test the security safeguards of your agent.

Location: `userspace/projects/task-02-vault/`

6.0.1 The Goal

Configure your agent so it is impossible for it to read sensitive files, specifically the `.env` file in this directory.

6.0.2 Steps

1. Navigate to `userspace/projects/task-02-vault/`. This directory contains a `.env` file with dummy credentials.
2. **The “Before” Check:** Ask the agent to read the contents of the `.env` file. It will likely succeed.
3. **Set up protection:**
 - *If using OpenCode:* Follow the [plugin instructions](#) to block `.env`.
 - *If using Gemini CLI:* Configure a hook to block `read_file` on `.env`.
4. **The “After” Check:** Ask the agent to read the `.env` file again. It should now be blocked!
5. **Programmatic Access:** Ask the agent: “How can I use the credentials in `.env` for my R script without you ever reading the file directly?” (Hint: `Sys.getenv()` or `dotenv` package).

Security First

In real research, never commit `.env` files to Git. Always ensure they are listed in your `.gitignore`.

7. The Workshop Workspace

Once your environment is ready, you will find a dedicated space for your work. Understanding the structure of this workspace is crucial for maintaining security and reproducibility.

7.1 Workspace Structure

The root directory contains several key folders:

- `userspace/`: This is your personal sandbox.
 - `userspace/examples/`: contains pre-built examples that we will explore together.
 - `userspace/projects/`: This is where you can start your own research projects or experiments.
- `data/`: Contains shared datasets for the workshop exercises.
- `tutorial-website-src/`: The source code for this website (if you want to see how it's built!).

7.2 The Sandbox Boundary

As we discussed in [Security & Sandboxing](#), the `userspace/` directory is designed to be your primary work area.

! Best Practice

When instructing an agent to create or modify files, always specify a path within `userspace/projects/`. This keeps your experimental code isolated from the core workshop materials.

7.3 Next Steps

Now that you understand where to work, let's learn how to [Interact with the Agents](#).

8. Environments & Skills

8.1 Managing Complex Dependencies

Agents often struggle when they encounter environments with missing dependencies, especially when packages require system-level libraries or compile from source.

We can solve this by: 1. **Prompting smartly:** Telling the agent *how* to install packages in our specific environment (e.g., using pre-compiled binaries). 2. **Creating Skills:** Saving that knowledge so the agent automatically knows how to handle the environment in the future.

Let's see how this works in a real, complex scenario.

[Go to Exercise 3: The Pro Resident](#) →

9. Exercise 3: The Pro Resident

Let's handle complex dependencies and teach the agent a permanent "Skill."

Location: `userspace/projects/task-03-demotools/`

9.0.1 The Goal

Install the `DemoTools` R package successfully in a Linux container without compiling from source, and create a reusable agent "Skill" for this process.

9.0.2 Steps

1. Navigate to `userspace/projects/task-03-demotools/`.
2. Ask the agent to run the `analysis.R` script. It will fail.
3. Prompt the agent to install `DemoTools`.
4. **Intervene:** If the agent tries to compile from source (it takes 30+ minutes), stop it. Guide it to find and install the *binary* version for Linux (e.g., via <https://timriffe.r-universe.dev>).
5. **Create a Skill:** Once the script runs, tell the agent: *"You've learned how to install binary packages on this system. Document this as a 'Skill' so you always prefer binaries over source compilation in this workspace."*
6. **Bonus:** Use the `HMD_USER` and `HMD_PASS` credentials (via environment variables) to fetch real data from the Human Mortality Database.

! Thinking in Code

This exercise demonstrates why we sometimes need to "override" the agent's default behavior with specific system knowledge.

References

This page lists the direct software, libraries, and local helper scripts used to build and display this book. It is intended as a practical attribution and reproducibility note, not a full software bill of materials for every transitive dependency bundled by each tool.

Production tools

Software	Version used locally	Role	License / project
Quarto	1.9.37	Builds the book website and PDF from the <code>.qmd</code> source files.	Open-source Quarto CLI
Pandoc	3.8.3	Document conversion engine used by Quarto.	GPL-2.0-or-later
Typst	0.14.2	PDF typesetting backend used by the Quarto <code>typst</code> format.	Apache-2.0

Website libraries

Software	How it is used	License / project
Typed.js 2.1.0	Loaded from unpkg.com for the animated “Beyond the Chatbox” title on the home page.	MIT
Bootstrap and the Cosmo theme	HTML layout, responsive navigation, typography, and theme styling through Quarto.	MIT
Bootstrap Icons	Icons used in Quarto navigation and controls.	MIT
Clipboard.js	Code-copy buttons generated by Quarto.	MIT
Fuse.js and autocomplete support	Client-side search generated by Quarto.	Apache-2.0
Tippy.js and Popper	Tooltips and popovers generated by Quarto.	MIT , MIT
AnchorJS	Header anchor links generated by Quarto.	MIT
Headroom.js	Navigation header behavior generated by Quarto.	MIT

i Typed.js license version

This book intentionally pins Typed.js to version 2.1.0, whose release tag includes an MIT license. The current `main` branch of Typed.js uses a GPL-3.0-or-later license. Re-check the upstream license before upgrading the CDN URL to a newer Typed.js release.

Local project code

The book also uses small local scripts and styles maintained in this repository:

File	Purpose
<code>styles.css</code>	Custom visual styling, including the Typed.js cursor styling.
<code>scripts/typed-init.js</code>	Initializes the Typed.js title animation.
<code>scripts/details-fold.js</code>	Adds behavior for collapsible details blocks.
<code>scripts/trigger-fix.lua</code> and <code>scripts/fix-typst-bg.lua</code>	Support the PDF/Typst rendering workflow.
<code>scripts/analytics.html</code>	Adds site analytics markup.

Content license

Unless otherwise noted, the workshop materials are distributed under [CC-BY-SA 4.0](#), as stated in the site footer. Third-party software, images and other content remains under their respective upstream licenses.